Learning outcomes:

1. selection sort, insertion sort, với mỗi thuật toán:
- biết cách chạy thuật toán với dữ liệu cụ thể (viết được vết chạy)
- biết độ phức tạp về thời gian chạy tính theo số phép so sánh, số phép đổi chỗ, trong các trường hợp tốt nhất, trung bình, tồi nhất
- có thể chọn thuật toán phù hợp với đặc điểm dữ liệu đầu vào (ví dụ mảng sắp ngược, đã sắp xếp, gần sắp xếp, nhiều khóa trùng nhau…)

2. thuật toán tráo ngẫu nhiên: hiểu ý tưởng, biết độ phức tạp

Demo các thuật toán sắp xếp: https://visualgo.net/en/sorting
Xem các thuật toán Insertion sort, Selection sort

Thử tự cài Selection Sort
Cho dãy số nguyên. Viết chương trình sắp xếp theo thứ tự tăng dần (mà không sử dụng hàm/thư viện có sẵn).
Ví dụ:
– Input: 1, 5, 2, 9, 8.
– Output: 1, 2, 5, 8, 9.
Gợi ý thuật toán:
Với i chạy từ đầu đến cuối dãy, mỗi lần :
chọn lấy phần tử có giá trị nhỏ nhất trong đoạn từ i đến cuối dãy
đổi chỗ số đó với phần tử tại vị trí i.

Quan sát thuật toán chạy với dữ liệu ngẫu nhiên, dữ liệu (gần) sắp xếp.
Có thể tham khảo code chương trình client Experiment trong slide 5 bài Elementary Sort. Đo thời gian bằng lệnh System.currentTimeMillis():

```
long start = System.currentTimeMillis();
//run something that takes time
long end = System.currentTimeMillis();
```

Bài 1 – Khảo sát Insertion Sort

Cho mã nguồn chương trình của thuật toán sắp xếp chèn (Insertion Sort) tại link mã nguồn môn học. Và dữ liệu trong bộ dữ liệu algs4-data.zip đã dùng từ các bài trước (bộ các file 1Kint.txt, 4Kint.txt….)
Hãy chạy lại các thuật toán trên (bạn có thể dùng thẳng mã nguồn đã cho hoặc viết lại phiên bản của riêng bạn theo thuật toán đã học trong bài giảng) với: (1) các file dữ liệu test, (2) dữ liệu sinh ngẫu nhiên, (3) dữ liệu đã được sắp xếp xuôi và (4) dữ liệu sắp xếp ngược, (5) dữ liệu toàn các giá trị bằng nhau. Với mỗi trường hợp cần đo thời gian chạy, với dữ liệu loại (1), (3) và (4) cần lấy trung bình của ít nhất 03 lần chạy, với dữ liệu loại (2) cần lấy trung bình của ít nhất 05 lần chạy. Với mỗi loại, chọn các kích thước dữ liệu khác nhau để thấy sự khác biệt về thời gian chạy. Thống kê thời gian chạy đối với từng loại dữ liệu từng kích thước dữ liệu. Nhận xét sự thay đổi của thời gian chạy theo dạng dữ liệu và kích thước dữ liệu.
Lưu ý, để đỡ tốn công viết code nhập và in dữ liệu, bạn nên dùng thư viện algs4 để đọc dữ liệu từ file. (lưu ý ký tự \ trong đường dẫn file phải viết thành \\ hai gạch chéo)

```
import edu.princeton.cs.algs4.*;
   ....
   public static void main(String[] args) {
     In in = new In("F:\\algs4-data\\4Kints.txt"); // tạo luồng đọc từ file
     int[] a = in.readAllInts();  // đọc toàn bộ file vào mảng a
     long start = System.currentTimeMillis();
     // xử lý dữ liệu trong mảng a
     long end = System.currentTimeMillis();  // thời gian chạy bằng end - start
     StdArrayIO.print(a); // in mảng ra màn hình
   }
   ....
```

Bài 2 – Khảo sát SelectionSort

Thực hiện tương tự với bài trên, thay Insertion Sort bằng Selection sort. Dùng cùng bộ dữ liệu với Insertion sort và so sánh thời gian chạy với Insertion Sort tại từng loại dữ liệu và kích thước dữ liệu

Bài 3 Insertion Sort – Part 1

Tóm tắt: Viết thêm vào thủ tục insertIntoSorted(int[] arr) để đưa phần tử cuối danh sách vào đúng vị trí để dãy tăng dần. Phần dãy trước đó (không kể phần tử cần tìm vị trí để chèn) đã được sắp. Cách chèn phần tử vào theo sắp xếp Chèn (Insertion sort). Tại mỗi bước in ra trạng thái mảng.
Đã có sẵn phần code nhập dữ liệu đầu vào, và in mảng ra printArray(int[] ar) (ở phía dưới).

Bài 4 Java Sort

Tóm tắt: Sắp xếp danh sách sinh viên theo trật tự lần lượt là: a) Điểm (CGPA) càng cao càng tốt, b) tên theo trật tự chữ cái và c) mã (id) càng nhỏ càng tốt.
(Đã có sẵn mã lệnh cho nhập dữ liệu vào và in ra. Chỉ cần viết phần sắp xếp.)
Gợi ý: Xem link: Sort ArrayList of custom Objects by property trên StackOverFlow. Trước tiên, cần tạo lớp giúp so sánh hai sinh viên, dạng:

```
class StudentComparator implements Comparator
{
   @Override
   public int compare(Student x, Student y)
   {
//Complete your code

return 0;
   }
}
```

Chú ý: Trong Java, so sánh xem String nào lớn hơn, dùng phương thức compareTo (vd: str1.compareTo(str2)).
Sau đó, trong chương trình chính, dùng phương thức Collections.sort() trong đó có sử dụng lớp so sánh đã tạo ở trên. Ví dụ:

```
Collections.sort(studentList, new StudentComparator());
```

Bài 5 Insertion Sort – Part 2

Tóm tắt: Viết thêm vào thủ tục insertionSortPart2(int[] ar) để sắp xếp theo thuật toán Insertion Sort.
Sau mỗi lần (vòng lặp) chèn được một số, in ra trạng thái mảng.
Đã có sẵn phần code nhập dữ liệu đầu vào, và in mảng ra printArray(int[] ar) (ở phía dưới).
Có thể sử dụng insertIntoSorted đã làm ở bài 3 trước vào bài này.
Ví dụ: thêm vào hàm đó tham số idx để giúp chèn vị trí tại idx vào danh sách [0 .. idx-1] trước nó.

Bài 6 Correctness Invariant

Tóm tắt: Trong thủ tục sắp xếp chèn insertionSort(int[] A) có một lỗi (error). Hãy tìm sửa lỗi đó (rồi submit code).

Bài 7 Counting Sort 1

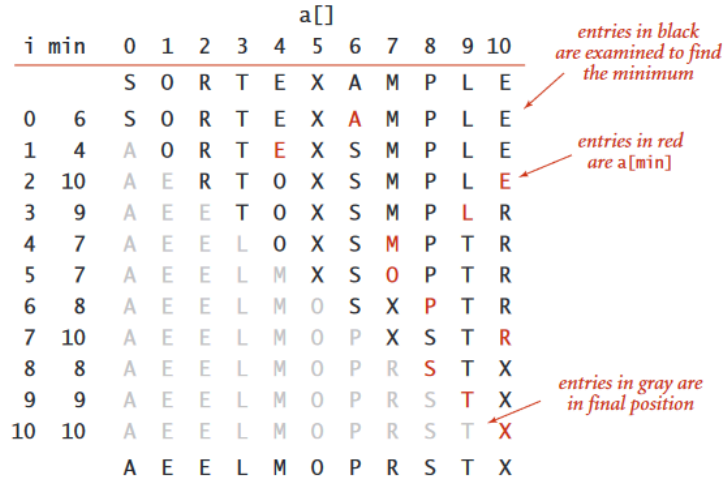
Tóm tắt: Cho danh sách các số nguyên. In ra (lần lượt) số lần xuất hiện của các số từ 0 tới 99.
Gợi ý: Không cần sắp xếp, chỉ cần đếm.

Bài 8. Sort và danh sách liên kết

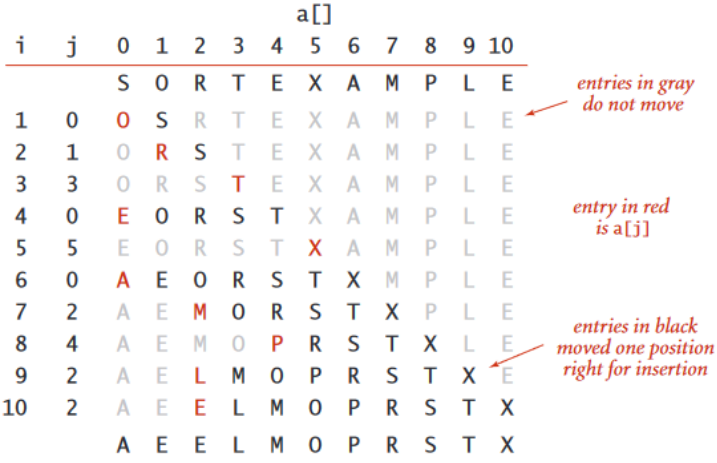
Trong hai thuật toán sắp xếp Chọn (Insertion sort) và Chèn/Xen vào (Selection Sort), dữ liệu đầu vào có nên lưu trong LinkedList (danh sách liên kết) không ?
Vì sao ?

Bài tập tại giáo trình **Algorithms 4th**

2.1.1 Trình bày cách sắp xếp chuỗi `E A S Y Q U E S T I O N` bằng thuật toán selection sort.
Gợi ý: trình bày tương tự ví dụ dưới đây. Các phần tử mầu đen được đọc giá trị để tìm giá trị min, các phần tử mầu đỏ là a[min], mầu xám là các phần tử đã được xếp vào đúng vị trí cuối cùng.



2.1.2 Trong thuật toán selection sort, một phần tử bất kì cần đến tối đa bao nhiêu lần đổi chỗ để được sắp vào đúng chỗ? Một phần tử bất kì cần đến trung bình bao nhiêu lần đổi chỗ để được sắp vào đúng chỗ?
2.1.3 Cho ví dụ về một mảng N phần tử mà khi chạy thuật toán selection sort có số lần kiểm tra `a[j] < a[min]` ra false (và vì thế cập nhật giá trị `min`) là tối đa.
2.1.4 Trình bày cách sắp xếp chuỗi `E A S Y Q U E S T I O N` bằng thuật toán insertion sort.
Gợi ý: trình bày tương tự ví dụ dưới đây. Các phần tử mầu đen di chuyển sang phải 01 vị trí để thực hiện việc chèn, phần tử mầu đỏ là a[j], mầu xám là các phần tử không di chuyển.



2.1.5 Với mỗi điều kiện trong hai điều kiện của vòng for bên trong của thuật toán insertion sort, hãy mô tả một mảng N phần tử mà điều kiện đó luôn false khi vòng lặp kết thúc.
2.1.6 Thuật toán nào chạy nhanh hơn đối với mảng gồm tất cả các key bằng nhau, selection sort hay insertion sort?
2.1.7 Thuật toán nào chạy nhanh hơn cho mảng đã sắp xếp thứ tự ngược, selection sort hay inser-

tion sort?

2.1.8 (*) Giả sử ta dùng thuật toán insertion sort đối với một mảng thứ tự ngẫu nhiên mà trong đó các phần tử chỉ nhận một trong 03 giá trị. Hỏi thời gian chạy sẽ là tuyến tính, bậc hai, hay cái gì đó ở giữa?